Heurística NEH + Búsqueda Local para NWJSSP

Oscar Díaz

Universidad EAFIT

3 de mayo de 2026

Contenido

1. Introducción
   - Descripción del Problema
   - Algoritmos
     - Algoritmo para la solución inicial
     - Algoritmo de Búsqueda Local
     - Algoritmo Variable Neighborhood Descent (VND)
     - Metaheurística: MultiStart + ELS + Recocido Simulado
2. Metodología
3. Resultados
   - Exposición de Resultados
   - Cota inferior
   - Búsqueda Local
   - VND y Metaheurístico
4. Discusion
   - Analisis resultados
   - Conclusiones
5. Referencias

Descripción del Problema

El **No-Wait Job Shop Scheduling Problem (NWJSSP)** consiste en programar un conjunto de n trabajos sobre un conjunto de m máquinas, minimizando el tiempo de flujo total $Z = \sum_j C_j$, donde C_j es el tiempo de finalización del trabajo j.

Cada trabajo j posee:

- Una **fecha de liberación** r_j: instante más temprano en que puede comenzar a procesarse.
- Una **secuencia fija de** m **operaciones**, donde la operación u requiere la máquina m_{ju} con un tiempo de procesamiento p_{ju}.

Las restricciones del problema son:

1. Cada operación u de un trabajo solo puede iniciar tras completar la operación $u - 1$.
2. Una vez iniciada, ninguna operación puede interrumpirse.
3. Cada máquina procesa a lo sumo un trabajo a la vez.
4. **Restricción No-Wait**: no se permite tiempo de espera entre operaciones consecutivas del mismo trabajo

Heurística Constructiva (NEH Adaptado)

El algoritmo constructivo implementado es una adaptación de la heurística **NEH** (Nawaz, Enscore y Ham, 1983) al NWJSSP.

Paso 1 Orden inicial

Para cada trabajo j se calcula el tiempo mínimo de procesamiento $r_j + \sum_u p_{ju}$, que combina su fecha de liberación con su carga total de procesamiento. Los trabajos se ordenan de **mayor a menor** por este tiempo mínimo de procesamiento, con el objetivo de que los trabajos que se demoran más tienen mayor prioridad a la hora de ser insertados.

Paso 2 Inserción iterativa

Los trabajos se procesan en el orden mencionado en Paso 1. Para cada trabajo j se busca la **mejor posición** dentro de la secuencia parcial ya construida, probando todas las posiciones disponibles $\{0, 1, \ldots, |\text{secuencia}|\}$.

Heurística Constructiva (NEH Adaptado)

Paso 3 Evaluación de posiciones por bloques temporizados
Evaluar todas las posiciones en instancias grandes era costoso y demorado computacionalmente. Para mejorar lo anterior, las posiciones se agrupan en **bloques de tamaño** $b = \text{máx}(10, \lfloor\sqrt{n}\rfloor)$. Cada bloque tiene un tiempo máximo de procesamiento $t_{\text{máx}}$: si se agota dentro del bloque, ese bloque se interrumpe y se avanza al siguiente. Todos los bloques son visitados siempre, garantizando cobertura global de la secuencia.

Paso 4 Criterio de evaluación
Para cada posición candidata se calcula el **Total Flow Time** $Z = \sum_j C_j$ que resultaría de insertar j allí, simulando la secuencia parcial completa.

Paso 5 Inserción
Al recorrer todos los bloques, el trabajo j se inserta en la posición de **menor impacto sobre** Z. El proceso se repite hasta que todos los trabajos han sido ubicados en la secuencia.

Búsqueda Local con Vecindario Swap

El algoritmo de búsqueda local con vecindario **Swap** se aplica sobre la solución inicial generada por la heurística NEH adaptada. Se implementaron dos criterios de selección diferentes:

Paso 1 Generación del vecindario Swap Dada una secuencia $B = [j_1, j_2, \ldots, j_n]$, se generan todos los vecinos intercambiando pares de trabajos. Para cada par de posiciones (i, j) con $i < j$, se crea un vecino P mediante el intercambio $B[i] \leftrightarrow B[j]$.

Paso 2a - First Improvement Se recorre el vecindario completo en el orden generado. Para cada vecino P se calcula su Total Flow Time $Z(P)$. En cuanto se encuentra un vecino con $Z(P) < Z(B)$, se acepta inmediatamente como nueva solución actual ($B \leftarrow P$), se actualiza $Z(B)$ y se reinicia la exploración del vecindario desde cero.

Búsqueda Local con Vecindario Swap

Paso 2b - Mixed Improvement De forma independiente, partiendo de la misma solución inicial NEH, se aplica el siguiente criterio:
Se genera el vecindario Swap completo. Luego se selecciona aleatoriamente un $R\,\%$ de los vecinos. Entre los vecinos muestreados se elige el de **mejor valor** $Z(P)$ (tipo Best Improvement parcial).
Si este mejor vecino muestreado mejora la solución actual ($Z(P) < Z(B)$), se acepta ($B \leftarrow P$) y se reinicia el proceso. En caso contrario, se detiene la iteración.

Paso 3 Condición de parada (ambos criterios) Ambos procedimientos se repiten mientras se encuentren mejoras y no se exceda el límite de tiempo de 3600 segundos (1 hora total compartido con NEH).

Búsqueda Local con Vecindario Insertion Up

El algoritmo de búsqueda local con vecindario **Insertion Up** se aplica de forma independiente sobre la solución inicial generada por la heurística NEH adaptada. Se implementaron los mismos dos criterios de selección que en el Swap: First Improvement y Mixed Improvement.

Paso 1 Generación del vecindario Insertion Up Dada una secuencia $B = [j_1, j_2, \ldots, j_n]$, se generan los vecinos moviendo un trabajo hacia una posición anterior. Para cada posición i y cada posición $j < i$, se crea un vecino P sacando el trabajo de la posición i e insertándolo en la posición j. Los pasos 2a (First Improvement), 2b (Mixed Improvement) y 3 (Condición de parada) son idénticos a los descritos en el algoritmo Swap.

Búsqueda Local con Vecindario Insertion Down

El algoritmo de búsqueda local con vecindario **Insertion Down** se aplica de forma independiente sobre la solución inicial generada por la heurística NEH adaptada. Se implementaron los mismos dos criterios de selección que en el Swap: First Improvement y Mixed Improvement.

Paso 1 Generación del vecindario Insertion Down Dada una secuencia $B = [j_1, j_2, \ldots, j_n]$, se generan los vecinos moviendo un trabajo hacia una posición posterior. Para cada posición i y cada posición $j > i$, se crea un vecino P sacando el trabajo de la posición i e insertándolo en la posición j. Los pasos 2a (First Improvement), 2b (Mixed Improvement) y 3 (Condición de parada) son idénticos a los descritos en el algoritmo Swap.

Variable Neighborhood Descent (VND)

El VND es una metaheurística de búsqueda local sistemática que utiliza múltiples estructuras de vecindario de forma secuencial.
Se aplica sobre la solución inicial generada por la **Heurística Constructiva NEH Adaptada** (ver sección anterior).
Características principales:

- Utiliza tres vecindarios: Insertion Down, Swap e Insertion Up.
- Emplea exclusivamente el criterio **First Improvement**.
- Cambia de vecindario al no encontrar mejoras.
- Reinicia desde el primer vecindario cada vez que se encuentra una mejora.

Algoritmo VND - First Improvement

Procedimiento:

- Iniciar con $k = 0$ (primer vecindario: Insertion Down)
- Aplicar **First Improvement** completo sobre el vecindario actual hasta que no se encuentren mejoras.
- Si se encontró una mejora:
  - Actualizar la solución actual
  - Reiniciar $k = 0$ (volver al primer vecindario)
- Si no se encontró mejora:
  - Pasar al siguiente vecindario ($k \leftarrow k + 1$)
- Detener cuando se hayan explorado los 3 vecindarios sin mejoras o se alcance el límite de tiempo (3600 segundos).

Metaheurística General

Multi-Start + ELS + Recocido Simulado

Algoritmo híbrido que combina:

- Construcción GRASP-NEH adaptada (Multi-Start)
- Búsqueda local con Recocido Simulado
- Perturbación
- Evolutionary Local Search

Búsqueda Local - Recocido Simulado

Vecindario: Insertion Backward (mover trabajo a posición anterior)
Parámetros:

- Temeperatura inicial $T_0 = 100$
- Temperatura final $T_F = 0{,}01$
- Factor de enfriamiento $ALPHA_COOLING = 0{,}1$
- Iteraciones por temperatura $L = 20$

Criterio: First Improvement o Probabilidad definida por la temperatura actual

Perturbación

Se aplica dentro del ELS para diversificar la búsqueda.

3 perturbaciones:

- 70 % probabilidad: **Inserción grande** (distancia ≥ 2)
- 30 % probabilidad: **Swap** aleatorio

Posteriormente se aplica Recocido Simulado completo sobre la solución perturbada.

Estructura General: Multi-Start + ELS

Fase Multi-Start ($N_{SOL} = 3$)

- 3 construcciones GRASP-NEH independientes (con $\alpha = 0{,}2$
- Cada una seguida de Recocido Simulado (temperatura baja)

Fase ELS ($N_{ITER_ELS} = 10$)

- Generar $N_{CANDIDATES} = 5$ perturbaciones
- Aplicar Recocido Simulado completo a cada una
- Seleccionar el mejor candidato
- Actualizar solución actual y global si mejora

Tiempo total límite: 3600 segundos (1 hora)

Metodología

1. **Implementación de algoritmos en Python:** Se programó tres algoritmos de búsqueda local (uno por vecindario: Swap, Insertion Up e Insertion Down), el VND y el metaheurístico híbrido todos aplicados al NWJSSP.
2. **Ejecución sobre instancias y exportación de resultados:** Cada uno de los cinco scripts recorre automáticamente las primeras 14 instancias disponibles y exporta los resultados en el formato requerido a archivos Excel separados (uno por criterio). Los algoritmos de busqueda local parten de la misma solución inicial NEH.
3. **Uso de cota inferior proporcionada:** Se utilizan directamente las cotas inferiores dadas por el profesor en el archivo `lb.txt` para calcular el GAP de cada solución.
4. **Comparación y análisis de resultados:** Se comparan el método constructivo NEH y los seis procedimientos de búsqueda local. También se comparan la variación de los parámetros en el Metaheurístico

Exposición de Resultados

Los resultados se presentan en el siguiente orden. Primero se muestra la **cota inferior teórica** (LB) utilizada para cada instancia, que sirve como referencia para medir la calidad de las soluciones. Luego se presentan los **resultados principales** de los ocho algoritmos, valor de la función objetivo Z y tiempo de cómputo t_c (en ms), junto con el **GAP** relativo respecto a la cota inferior:

$$\text{GAP} = \frac{Z - LB}{LB}$$

Finalmente se presentan los **experimentos de parámetros** fueron realizados para las primeras seis y ocho instancias por coste computacional: para cada método de busqueda local se varía el parámetro MIXED R, con el objetivo de identificar la configuración que produce los mejores valores de Z. Mientras que para el metaheurístico híbrido se cambian diferentes parámetros.

Cotas inferiores

Índice	Instancia	n	m	LB
1	ft06	6	6	259
2	ft06r	6	6	283
3	ft10	10	10	7377
4	ft10r	10	10	6318
5	ft20	20	5	14139
6	ft20r	20	5	13257
7	tai_j10_m10_1	10	10	64463
8	tai_j10_m10_1r	10	10	61293
9	tai_j100_m10_1	100	10	2939670
10	tai_j100_m10_1r	100	10	2827631
11	tai_j100_m100_1	100	100	5637873
12	tai_j100_m100_1r	100	100	6146561
13	tai_j1000_m10_1	1000	10	259534722
14	tai_j1000_m10_1r	1000	10	258454996

Búsqueda Local - GAP respecto a la cota inferior

Instancia	Swap-FI	Swap-M	InsUP-FI	InsUP-M	InsDOWN-FI	InsDOWN-M
1	0.236	0.355	0.189	0.405	0.212	0.247
2	0.233	0.286	0.134	0.208	0.191	0.371
3	0.360	0.548	0.373	0.425	0.406	0.414
4	0.643	0.852	0.641	0.822	0.741	0.888
5	0.294	0.398	0.272	0.293	0.315	0.323
6	0.344	0.447	0.390	0.422	0.503	0.512
7	0.684	0.769	0.724	0.739	0.742	0.766
8	0.611	0.720	0.641	0.815	0.804	0.867
9	1.141	1.177	1.203	1.192	1.238	1.212
10	1.212	1.242	1.296	1.257	1.305	1.344
11	6.835	6.770	6.873	6.889	6.847	7.081
12	6.106	6.057	6.202	6.175	6.163	6.432
13	1.255	1.247	1.239	1.275	1.258	1.261
14	1.257	1.255	1.240	1.231	1.245	1.256
GAP promedio	**1.515**	**1.580**	**1.530**	**1.582**	**1.569**	**1.641**

Búsqueda Local - Swap: variación de R

Instancia	FI	MI 0.1	MI 0.2	MI 0.4	MI 0.6	MI 0.8
1	0.236	0.375	0.367	0.212	0.189	0.212
2	0.233	0.371	0.134	0.145	0.191	0.191
3	0.360	0.442	0.479	0.373	0.428	0.414
4	0.643	0.999	0.782	0.758	0.615	0.643
5	0.294	0.396	0.396	0.339	0.273	0.294
6	0.344	0.443	0.733	0.756	0.357	0.382
7	0.684	0.854	0.857	0.678	0.669	0.669
8	0.611	0.763	0.824	0.664	0.641	0.810
GAP promedio	**0.425**	**0.579**	**0.572**	**0.491**	**0.421**	**0.454**

Búsqueda Local - Insertion UP: variación de R

Instancia	FI	MI 0.1	MI 0.2	MI 0.4	MI 0.6	MI 0.8
1	0.189	0.448	0.405	0.193	0.189	0.189
2	0.134	0.371	0.134	0.191	0.191	0.191
3	0.373	0.454	0.365	0.414	0.414	0.414
4	0.641	0.838	0.920	0.772	0.697	0.697
5	0.272	0.402	0.296	0.307	0.270	0.240
6	0.390	0.496	0.443	0.458	0.440	0.440
7	0.724	0.713	0.743	0.588	0.555	0.560
8	0.641	0.871	0.726	0.810	0.682	0.641
GAP promedio	**0.421**	**0.574**	**0.504**	**0.492**	**0.430**	**0.422**

Búsqueda Local - Insertion DOWN: variación de R

Instancia	FI	MI 0.1	MI 0.2	MI 0.4	MI 0.6	MI 0.8
1	0.212	0.247	0.247	0.212	0.189	0.212
2	0.191	0.371	0.191	0.145	0.191	0.191
3	0.406	0.424	0.508	0.508	0.406	0.406
4	0.741	0.832	0.909	0.855	0.833	0.800
5	0.315	0.365	0.281	0.343	0.277	0.264
6	0.503	0.544	0.448	0.433	0.443	0.433
7	0.742	0.679	0.854	0.515	0.515	0.711
8	0.804	0.757	0.825	0.843	0.825	0.671
GAP promedio	**0.489**	**0.527**	**0.533**	**0.482**	**0.460**	**0.461**

VND y Metaheurístico - GAP respecto a la cota inferior

Instancia	VND	Metaheuristico
1	0.212	0.189
2	0.134	0.134
3	0.330	0.340
4	0.595	0.615
5	0.275	0.287
6	0.375	0.422
7	0.531	0.558
8	0.729	0.611
9	1.230	1.388
10	1.320	1.480
11	7.026	6.805
12	6.293	6.590
13	1.251	1.240
14	1.239	1.234
GAP promedio	**1.539**	**1.564**

Metaheurístico - Variación del parámetro *ALPHA_COOLING*

Instancia	*ALPHA_COOLING* = 0,9		*ALPHA_COOLING* = 0,5		*ALPHA_COOLING* = 0,1	
	GAP	dt	GAP	dt	GAP	dt
1	0.189	144034	0.189	3354	0.189	0
2	0.134	48068	0.134	3050	0.134	0
3	0.330	3436499	0.343	411546	0.340	0
4	0.616	1364737	0.595	378124	0.615	0
5	0.245	3379535	0.249	1535744	0.287	0
6	0.378	3361774	0.407	535836	0.422	0
Promedio	**0.315**	**1955775**	**0.320**	**477942**	**0.331**	**0**

dt = tiempo adicional (ms) respecto a la configuración más rápida

Metaheurístico - Variación de $N_{candidates}$

Instancia	$N_{cand} = 3$		$N_{cand} = 5$		$N_{cand} = 10$	
	GAP	dt	GAP	dt	GAP	dt
1	0.189	144034	0.189	3354	0.189	0
2	0.134	48068	0.134	3050	0.134	0
3	0.330	3436499	0.330	411546	0.340	0
4	0.620	1364737	0.616	378124	0.595	0
5	0.251	3379535	0.245	1535744	0.227	0
6	0.403	3361774	0.378	535836	0.326	0
Promedio	**0.321**	**0**	**0.315**	**1955775**	**0.302**	**152287**

dt = tiempo adicional (ms) respecto a la configuración más rápida

Metaheurístico - Variación de N_{iter}

Instancia	$N_{iter} = 3$		$N_{iter} = 5$		$N_{iter} = 10$	
	GAP	dt	GAP	dt	GAP	dt
1	0.189	144034	0.189	3354	0.189	0
2	0.145	48068	0.134	3050	0.134	0
3	0.343	3436499	0.330	411546	0.330	0
4	0.631	1364737	0.595	378124	0.616	0
5	0.243	3379535	0.234	1535744	0.245	0
6	0.403	3361774	0.415	535836	0.378	0
Promedio	**0.326**	**0**	**0.316**	**15292**	**0.315**	**1955775**

dt = tiempo adicional (ms) respecto a la configuración más rápida

Metaheurístico - Variación de N_{per}

Instancia	$N_{per} = 3$		$N_{per} = 5$		$N_{per} = 10$	
	GAP	dt	GAP	dt	GAP	dt
1	0.189	144034	0.189	3354	0.189	0
2	0.134	48068	0.134	3050	0.134	0
3	0.330	3436499	0.340	411546	0.342	0
4	0.616	1364737	0.626	378124	0.616	0
5	0.245	3379535	0.255	1535744	0.278	0
6	0.378	3361774	0.437	535836	0.408	0
Promedio	**0.315**	**1955775**	**0.330**	**4595**	**0.328**	**622**

dt = tiempo adicional (ms) respecto a la configuración más rápida

Metaheurístico - Variación de N_{sol}

Instancia	$N = 3$		$N = 5$		$N = 10$	
	GAP	dt	GAP	dt	GAP	dt
1	0.189	144034	0.189	3354	0.189	0
2	0.134	48068	0.134	3050	0.134	0
3	0.330	3436499	0.330	411546	0.330	0
4	0.616	1364737	0.616	378124	0.595	0
5	0.245	3379535	0.241	1535744	0.238	0
6	0.378	3361774	0.376	535836	0.346	0
Promedio	**0.315**	**1955775**	**0.314**	**87490**	**0.305**	**666075**

dt = tiempo adicional (ms) respecto a la configuración más rápida

Análisis de resultados - Búsqueda Local

- El algoritmo con mejor GAP promedio fue **Swap-FI** (1.515), seguido por InsUP-FI (1.530) e InsDOWN-FI (1.569). En general, los métodos con criterio **First Improvement** superaron a sus versiones Mixed en los tres vecindarios evaluados.
- En los experimentos de parámetros (realizados sobre las primeras 8 instancias), se observó que valores de $0{,}4 \leq MIXED_R \leq 0{,}8$ permiten obtener GAP promedio competitivos e incluso inferiores al criterio First Improvement en algunos casos.

Análisis de Resultados - VND y Metaheurístico

- El VND obtuvo un GAP promedio de **1.539**, mientras que el Metaheurístico alcanzó un GAP promedio de **1.564**.
- El VND presenta un desempeño ligeramente mejor en promedio, aunque el Metaheurístico logra mejores resultados en algunas instancias de mayor tamaño.

Análisis de Experimentos - VND y Metaheurístico

- Se observó que valores bajos de *ALPHA_COOLING* (por ejemplo, 0,1) producen mejores GAP promedio y menores tiempos de ejecución.
- Incrementar $N_{candidates}$ y N_{iter_ELS} tiende a mejorar la calidad de las soluciones, aunque con un aumento significativo en el tiempo computacional.
- Los parámetros N_{per} y N_{sol} presentan menor impacto en el GAP, pero influyen en el tiempo total y en la diversidad de soluciones exploradas.

Conclusiones

- La combinación **NEH + Búsqueda Local Swap con First Improvement** presenta el mejor desempeño global, logrando el menor GAP promedio. Esto sugiere que un vecindario simple con exploración rápida de mejoras locales es altamente efectivo bajo restricciones de tiempo.
- Aunque el criterio First Improvement domina en promedio, los resultados muestran que configuraciones de Mixed Improvement con $0{,}4 \leq MIXED_R \leq 0{,}8$ pueden ofrecer un mejor balance entre exploración y explotación, permitiendo encontrar soluciones de alta calidad sin recorrer completamente el vecindario.
- El algoritmo VND demuestra ser una estrategia robusta al combinar múltiples vecindarios de forma sistemática, logrando resultados consistentes en distintas instancias.
- El Metaheurístico propuesto, al incorporar diversificación (GRASP, perturbaciones) e intensificación (Recocido Simulado), muestra mayor capacidad de explorar el espacio de soluciones, especialmente en instancias grandes, aunque su desempeño promedio no supera al VND

Referencias

Nawaz, M., Enscore, E. E., & Ham, I. (1983).
A heuristic algorithm for the *m*-machine, *n*-job flow-shop sequencing problem.
Omega, 11(1), 91–95.

Feo, T. A., & Resende, M. G. C. (1989).
A probabilistic heuristic for a computationally difficult set covering problem.
Operations Research Letters, 8(2), 67–71.